

Séance 4

Aujourd'hui

- Modèle de classe *vector*
- Chaîne de caractères

Modèle de classe

- Problème
 - La classe TableauInt ne gère que des éléments entiers
 - Il faudrait faire d'autres classes :
 - TableauDouble, TableauFloat, TableauEtudiant,...
- Solution pratique
 - Utiliser un modèle de classe
 - Mettre le type des éléments en « paramètre »

Utilisation d'un modèle

- Avec la même classe Tableau, on peut écrire
Tableau<int> ti(10); // version int du modèle
Tableau<double> td(20); // version double
Tableau<short> ts(20); // version short
Tableau<Etudiant> te(5) // tableau d'étudiants

Mise en oeuvre

- Déclaration d'un modèle de classe (template)

```
template <class T> class Tableau
{
    T * tableau_;
    int  taille_;
public :
    Tableau(int taille=0);
    ~Tableau();
    int getTaille() const {return taille_;}
    T & operator[](int i);
};
```

Définition des méthodes

```
template <class T> Tableau<T>::Tableau(int taille)
{
    if (taille<0) taille=0;
    taille_=taille;
    tableau_=new T[taille_];
}
```

```
template <class T> Tableau<T>::~~Tableau()
{
    delete [ ] tableau_;
}
```

Le modèle *vector*

- Il existe un modèle de classe "tableau" appelé *vector*
 - Il est défini dans la librairie standard du C++ : STL
 - Il sera utilisé dans la suite du cours, les exercices et les contrôles
 - Il possède les fonctionnalités d'un tableau dynamique

Standard Template Library (STL)

- Bibliothèques standardisée de classes et de fonctions
- Basée sur l'utilisation des modèles
- Contient
 - Structures de données (conteneurs)
 - vector, list, map,...
 - Algorithmes
 - recherche, tri, insertions, suppressions
 - Classe string
 - Manipulation des chaînes de caractères

Utilisation de vector (1/2)

- Constructeur

```
vector<int> t1;           // Tableau d'entiers vide
```

```
vector<double> t2(5);    // Tableau de 5 réels à 0
```

```
vector<int> t3(10,2);    // Tableau de 10 entiers à 2
```

- Taille

```
cout<<"Taille de t1 : "<<t1.size();
```

```
t1.resize(5);           // Changement de taille
```

- Accès aux éléments

```
for(int i=0;i<t1.size();i++) t1[i]=4;
```

Utilisation de vector (2/2)

- Affectation et copie

```
t1=t3;
```

```
vector<int> t4(t1);
```

- Ajout et retrait à la fin

```
t1.push_back(3); // Ajout en dernier
```

```
t3.pop_back();   // Retrait en dernier
```

Principales méthodes du modèle vector (1/3)

- `vector( ) ;`
 - Constructeur sans paramètres (vecteur de taille 0)
- `vector( int n, const T& value= T() ) ;`
 - Constructeur avec taille (n) et valeurs d'initialisation (value)
- `vector( const vector<T>& x ) ;`
 - Constructeur de copie
- `unsigned size( ) const ;`
 - Retourne la taille du vecteur
- `void resize( int n, T& value= T( ) )`
 - Changement de taille, les éléments ajoutés sont initialisés avec la valeur donnée en paramètres

Principales méthodes du modèle vector (2/3)

- `vector<T>& operator= (const vector<T >& x);`
 - Opérateur d'affectation : les 2 vecteurs deviennent identiques
- `T& operator[] ( int i ) ;`
 - Accès à l'élément d'indice i en lecture ou en écriture
- `T& at(int i) ;`
 - Accès aux éléments avec vérification de l'indice
- `T& back() ;`
 - Accès au dernier élément
- `T& front() ;`
 - Accès au premier élément

Principales méthodes du modèle vector (3/3)

- `void push_back ( const T& x );`
 - Ajout d'un nouvel élément de valeur x à la fin
- `void pop_back ( );`
 - Suppression du dernier élément
- `void clear ( );`
 - Suppression de tous les éléments

Exemple d'utilisation (1/2)

```
// Fichier main.cpp
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    vector<double> tab;
    double note;
    do
    {
        cout<<"Entez une note (-1 pour terminer) : ";
        cin>>note;
        tab.push_back(note);
    }
    while (note>0);
    tab.pop_back();
}
```

Exemple d'utilisation (2/2)

```
cout<<"Vous avez entré "<<tab.size()<<" notes"<<endl;
cout<<"Voici vos notes :";
for (int i=0 ;i<tab.size();i++) cout<<tab[i]<<" ";
cout<<endl;

double moyenne=0;
for (int i=0;i<tab.size();i++) moyenne+=tab[i];
moyenne=moyenne/tab.size();
cout<<"Moyenne :"<<moyenne<<endl;

vector<double> tab2(10);
cout<<"tab2 :";
for (int i=0 ;i<tab2.size();i++) cout<<tab2[i]<<" ";
cout<<endl;
tab2=tab;    // Copie des notes dans tab2
cout<<"tab2 après affectation :";
for (int i=0 ;i<tab2.size();i++) cout<<tab2[i]<<" ";
cout<<endl;
return 0 ;
}
```

Remarque sur les tableaux d'objets

- Si l'on veut déclarer un tableau d'objet, le constructeur sans paramètre doit exister.
- Soit la classe

```
class Etudiant
{
    char nom_[80] ;
public :
    Etudiant(const char * nom) ;
    void afficher() ;
} ;
Etudiant::Etudiant(const char * nom)
{
    strcpy(nom_, nom) ;
}
void Etudiant::afficher()
{ cout<<nom_<<endl ; }
```


Déclaration d'un tableau d'objets

- Les déclarations :

Etudiant tab1[10] ;

vector<Etudiant> tab2(10) ;

- Produisent une erreur de compilation car elle réalisent :
 - La déclaration d'un tableau
 - L'appel du constructeur pour chaque élément
- Il faut que le constructeur puisse être appelé sans paramètres.

Pour que cela fonctionne

- Il faut ajouter un constructeur sans paramètres ou une valeur d'initialisation par défaut :

```
class Etudiant
{
    char nom_[80] ;
public :
    Etudiant(const char * nom="") ;
    void afficher() ;
} ;
Etudiant::Etudiant(const char * nom)
{
    strcpy(nom_,nom) ;
}
void Etudiant::afficher()
{ cout<<nom_<<endl ; }
```

Aujourd'hui

- Modèle de classe *vector*
- Chaîne de caractères

Gestion d'une chaîne en C

- Représentation d'une chaîne
 - Chaîne = tableau de caractères terminé par 0

b	o	n	j	o	u	r	\0
0	1	2	3	4	5	6	7

Pour manipuler des chaînes :

- Déclaration d'un tableau de caractère

```
char ch1[8]= "bonjour";  
char ch2[8];
```
- Appel de fonctions spécifiques (strlen, strcpy,...)

```
strcpy(ch2,ch1);
```

Pourquoi une classe "chaîne" ?

- Simplifier les manipulations
- Gestion dynamique
- Exemples

```
string ch1="bonjour";
```

```
string ch2;
```

```
ch2=ch1;           // copie
```

```
ch2=ch2+" monsieur"; // concaténation
```

```
cout<<ch2;
```

Construction d'une classe "chaîne"

- Même principe que pour la classe *TableauInt*
 - Gestion d'une chaîne dynamique par une classe "chaîne"
 - Ajout de méthodes spécifiques à la manipulation des chaînes

La classe *string* de la STL

- Il existe une classe "chaîne" appelée *string*
 - Elle est définie dans la librairie standard du C++
 - Elle sera utilisée dans la suite du cours, les exercices et les contrôles

Utilisation (1/3)

- Construction

```
string s1;    // Constructeur par défaut
string s2("Bonjour"); // A partir d'une constant chaîne
string s3("Bonjour",3); // Seulement : "Bon"
string s4(s2); // Constructeur de copie
string s5(s2,3,4); // Une partie seulement : "jour"
string s6(5,'A'); // "AAAAA"
```

- Taille

```
cout<<"Taille de s1 :"<<s1.size()<<endl;
s3.resize(6,'-'); // "Bon---"
```


Utilisation (2/3)

- Copie, concaténation, sous-chaîne

```
string s1("Bon");
```

```
string s2,s3;
```

```
s2=s1+string("jour");    // ou s2=s1+"jour";
```

```
s3=s2.substr(1,4);       // "onjo"
```

- Recherche

```
cout<<"position : "<<s2.find("jour");    // 3
```

```
cout<<"position : "<<s2.find_first_of("aeiouy"); // 1
```

Utilisation (3/3)

- Saisie sur *cin*

```
string s1,s2,s3;  
cout<<"Entrer une chaîne sans espaces"<<endl;  
cin>>s1;  
cout<<"Votre chaîne : "<<s1<<endl;
```

```
cout<<"Entrer une chaîne avec espaces"<<endl;  
getline(cin,s2);  
cout<<"Votre chaîne : "<<s2<<endl;
```

```
cout<<"Entrer une chaîne terminée par #"<<endl;  
getline(cin,s3,'#');  
cout<<"Votre chaîne : "<<s3<<endl;
```

Principales méthodes de la classe string (1/4)

- `string( ) ;`
 - Constructeur sans paramètres (chaine vide)
- `string ( const char *p);`
 - Constructeur avec une constante chaine du type "C"
- `string ( const char *p, int n);`
 - Idem mais seulement n éléments sont utilisés
- `string ( const char *p, int n);`
 - Constructeur de copie
- `string ( int n,char c);`
 - Constructeur : chaine de taille n initialisée avec c

Principales méthodes de la classe string (2/4)

- `unsigned size() const;`
 - Retourne le nombre de caractères
- `unsigned lenght() const;`
 - Identique à `size( )`
- `void resize ( int n, char c = '\0');`
 - Change la taille, les caractères ajoutés sont initialisés avec `c`
- `string& operator= (const string& s);`
 - Opérateur d'affectation

Principales méthodes de la classe string (3/4)

- `char& operator[] ( int i ) ;`
 - Accès au caractère d'indice i (affectation ou lecture)
- `char& at(int n) ;`
 - Idem avec accès sécurisé
- `string& operator+=(const string &s) ;`
 - Concatène s à la fin de la chaîne courante
- `void push_back ( char c ) ;`
 - Ajout d'un caractère à la fin
- `const char* c_str();`
 - Retourne l'adresse d'une constant chaîne du "C" terminée par nul

Principales méthodes de la classe `string` (4/4)

- `unsigned find(const string& s, int pos=0);`
 - Cherche la première occurrence de la sous-chaine identique à `s` à partir de la position `pos` et retourne son indice. Si la chaine n'est pas trouvée, retourne `string::npos`. La fonction *rfind* fait recherche en partant de la fin.
- `int find_first_of(const string &s, int pos=0);`
 - Cherche le premier caractère égal à l'un des caractères de `s` et retourne sa position (ou `string::npos`). Il existe aussi *find_last_of*, *find_first_not_of*, *find_last_not_of*.
- `string substr(int pos=0, int n=npos);`
 - Retourne une sous chaîne commençant à la position `pos` et contenant au plus `n` caractères.

Autres fonctions et opérateurs

- string operator +(const string& s1, const string& s2);
 - Concaténation
- bool operator <(const string& s1, const string& s2);
 - Comparaison (il existe aussi >, <=, >=, !=, ==)
- ostream& operator<< (ostream& os, const string& str);
 - Opérateur d'injection sur un flux de sortie (cout)
- istream& operator>> (istream& is, string& str);
 - Opérateur d'extraction à partir d'un flux (cin)
- istream& getline(istream& is, string& s, char delim='\n') ;
 - Extraction à partir d'un flux jusqu'au caractère séparateur

Exemple d'utilisation (1/3)

```
// Fichier main.cpp

#include <iostream>
#include <string>

using namespace std;

int main()
{
    string nom ;
    cout<<"Nom de login : ";
    cin>>nom;    // opérateur >>
```


Exemple d'utilisation (2/3)

```
string password("toto");
bool ok=false;
do
{
    string p ;
    cout<<"Mot de passe : ";
    cin>>p;
    if (p!=password)    // Comparaison de chaînes
    {
        cout<<"Mot de passe incorrect"<<endl;
    }
    else ok=true;
} while(!ok);
```

Exemple d'utilisation (3/3)

```
string message ;
message="Bonjour "+nom; // Concaténation de chaînes
cout<<message<<endl; // opérateur <<
do
{
    string p ;
    cout<<"Entrez un nouveau mot de passe : ";
    cin>>p;
    if (p.size()<5)    // Taille de la chaîne
        cout<<"mot de passe trop court"<<endl;
    } while(p.size()<5)
password=p; // Copie de chaînes

return 0 ;
}
```

Exercice 4

Dans cet exercice, on utilisera la classe `string` de la STL (questions a,b,c) et le modèle `vector` (question c).

a) Ecrire un programme qui demande une phrase à l'utilisateur et stocke cette phrase dans un objet `string`.

b) A partir des méthodes *`find_first_of`* et *`substr`*, écrire une boucle qui affiche un par un les mots de la chaîne (on supposera que les mots sont séparés par un caractère espace).

Exercice 4 (suite)

c) Transformer le code de la question b) en une fonction qui va remplir un `vector<string>` avec les mots de la phrase. La déclaration de la fonction sera :

```
void extraireMots(const string & phrase, vector<string> & mots) ;
```

d) Adapter le programme de test pour qu'il appelle la fonction et affiche les mots du tableau de sortie.